

CISD 43 - Template- Code

June 7, 2026

0.1 NREL Supply Solar Supply Curves 2030

```
[1]: # Edit all the Mardown cells below with the appropriate information
# Run all cells, containing your code
# Save this Jupyter with the outputs of your executed cells
# PS: Save again the notebook with this outcome.
# PSPS: Don't forget to include the dataset in your submission
```

Team: * Christina Perez DeHaro

Course: CISD 43 – BIG DATA (Spring, 2026)

1 Problem Statement

- My project aims to analyze the NREL Limited Access 2030 Moderate PV Supply Curve dataset that is produced by NREL's reV (Renewable Energy Potential) Models. The goal is to identify patterns in solar development costs across California and evaluate which site characteristics most influence LCOE (Levelized Cost Of Energy).
- **Keywords:** Linear Regression, NREL, KNN, NREL Limited Access 2030 Moderate PV Supply Curves

1.1 Required packages

- Add instructions to install the required packages

```
[15]: #! pip install pymongo
```

```
[17]: # Run on correct numpy version if necessary
# import subprocess
# subprocess.run(["pip", "install", "numpy<2", "--quiet"])
```

```
[19]: #Import the Necessary Libararies Packages
```

```
import pandas as pd
from pymongo import MongoClient
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

1.1.1 Methodology

1. Data is ingested and stored in MongoDB, followed by Exploratory Data Analysis (EDA) to profile LCOE distributions across California counties. Two machine learning models are then trained and evaluated
2. Introduce the topics you used in your project
 - Model 1
 - Linear Regression
 - Model 2
 - KNN

1.2 Connect to dataset hosted in MongoDB

```
[23]: client = MongoClient('mongodb://localhost:27017/')
```

```
[25]: db = client["testing"]
```

```
[27]: collection = db["nrel"]
```

```
[29]: # Retrieve the first three records
result = collection.find().limit(5)

# Print the records
for record in result:
    print(record)
```

```
{'_id': ObjectId('6a23b308710edabdd297a94a'), 'area_sq_km': 1.204307940673828,
'capacity_mw_ac': 38.64570449377158, 'capacity_mw_dc': 51.7852414489746,
'cnty_fips': 53073, 'country': 'United States', 'county': 'Whatcom', 'dist_km':
0.9963961030678928, 'elevation': 23.2, 'latitude': 48.994, 'lcot':
41.386483384381776, 'longitude': -122.735, 'mean_cf_ac': 0.19883126,
'mean_cf_dc': 0.14838155, 'mean_lcoe': 59.87425709754124, 'mean_res': 3.377832,
'reg_mult': 1.0187303, 'reinforcement_cost_per_mw_ac': 813613.0513085702,
'reinforcement_dist_km': 229.4184349193356, 'sc_point_gid': 416, 'state':
'Washington', 'timezone': -8, 'total_lcoe': 101.26074048192302,
'trans_cap_cost_per_mw_ac': 77430.2111570799}
{'_id': ObjectId('6a23b308710edabdd297a94b'), 'area_sq_km': 0.8135640197753906,
'capacity_mw_ac': 26.10690640918063, 'capacity_mw_dc': 34.9832528503418,
'cnty_fips': 53055, 'country': 'United States', 'county': 'San Juan', 'dist_km':
14.00491557497398, 'elevation': 32.8, 'latitude': 49.022, 'lcot':
59.52512619781256, 'longitude': -122.575, 'mean_cf_ac': 0.1956991, 'mean_cf_dc':
0.1460441, 'mean_lcoe': 60.68953568115725, 'mean_res': 3.3234246, 'reg_mult':
1.015822, 'reinforcement_cost_per_mw_ac': 776337.8233968806,
'reinforcement_dist_km': 207.5515388290221, 'sc_point_gid': 417, 'state':
'Washington', 'timezone': -8, 'total_lcoe': 120.2146618789698,
'trans_cap_cost_per_mw_ac': 485038.58143157937}
{'_id': ObjectId('6a23b308710edabdd297a94c'), 'area_sq_km': 0.1613520006179809,
'capacity_mw_ac': 5.177713709894209, 'capacity_mw_dc': 6.93813602657318,
```

```

'cnty_fips': 53073, 'country': 'United States', 'county': 'Whatcom', 'dist_km':
7.467350647362943, 'elevation': 20.0, 'latitude': 48.873, 'lcot':
355.0788057193065, 'longitude': -122.848, 'mean_cf_ac': 0.20407446,
'mean_cf_dc': 0.15229434, 'mean_lcoe': 58.3531232518396, 'mean_res': 3.465,
'reg_mult': 1.0190946, 'reinforcement_cost_per_mw_ac': 806946.6543928104,
'reinforcement_dist_km': 216.31991544742945, 'sc_point_gid': 795, 'state':
'Washington', 'timezone': -8, 'total_lcoe': 413.4319289711461,
'trans_cap_cost_per_mw_ac': 7039424.995077247}
{'_id': ObjectId('6a23b308710edabdd297a94d'), 'area_sq_km': 3.37365,
'capacity_mw_ac': 108.2589232887268, 'capacity_mw_dc': 145.06695, 'cnty_fips':
53073, 'country': 'United States', 'county': 'Whatcom', 'dist_km':
0.9436753236814714, 'elevation': 40.5625, 'latitude': 48.9, 'lcot':
38.12626669097469, 'longitude': -122.688, 'mean_cf_ac': 0.20262648,
'mean_cf_dc': 0.1512138, 'mean_lcoe': 58.75952227749448, 'mean_res': 3.4343858,
'reg_mult': 1.0188717, 'reinforcement_cost_per_mw_ac': 804270.6694975446,
'reinforcement_dist_km': 216.0435193443616, 'sc_point_gid': 796, 'state':
'Washington', 'timezone': -8, 'total_lcoe': 96.88578896846916,
'trans_cap_cost_per_mw_ac': 32248.83223733312}
{'_id': ObjectId('6a23b308710edabdd297a94e'), 'area_sq_km': 0.9396000617980956,
'capacity_mw_ac': 30.15134318027767, 'capacity_mw_dc': 40.40280265731811,
'cnty_fips': 53073, 'country': 'United States', 'county': 'Whatcom', 'dist_km':
6.911909088590098, 'elevation': 28.75, 'latitude': 48.927, 'lcot':
47.129721244095805, 'longitude': -122.528, 'mean_cf_ac': 0.1966125,
'mean_cf_dc': 0.14672574, 'mean_lcoe': 60.423369773344085, 'mean_res': 3.343731,
'reg_mult': 1.0161444, 'reinforcement_cost_per_mw_ac': 776337.8233968806,
'reinforcement_dist_km': 207.5515388290221, 'sc_point_gid': 797, 'state':
'Washington', 'timezone': -8, 'total_lcoe': 107.5530910174399,
'trans_cap_cost_per_mw_ac': 227033.287301512}

```

2 EDA

2.1 Create dataframe for data within the state of California

```

[33]: query = {'state': "California"}

result = collection.find(query)

# Convert the result to a pandas DataFrame
df = pd.DataFrame(list(result))

# Display the DataFrame
print(df)

```

	_id	area_sq_km	capacity_mw_ac	capacity_mw_dc	\
0	6a23b30b710edabdd297d8ea	0.203472	6.529325	8.749296	
1	6a23b30b710edabdd297da12	0.684612	21.968892	29.438317	
2	6a23b30b710edabdd297da13	1.606716	51.558794	69.088788	
3	6a23b30b710edabdd297da14	1.082808	34.746824	46.560747	

4	6a23b30b710edabdd297da15	0.016524	0.530248	0.710532
...	...	...	...	...
2755	6a23b311710edabdd2985cf7	65.553945	2103.596587	2818.819621
2756	6a23b311710edabdd2985cf8	59.255064	1901.468099	2547.967732
2757	6a23b311710edabdd2985df7	2.258280	72.467186	97.106036
2758	6a23b311710edabdd2985ef5	13.585076	435.939039	584.158284
2759	6a23b311710edabdd2985fec	3.486564	111.882284	149.922253

	cnty_fips	country	county	dist_km	elevation	latitude	\
0	6015	United States	Del Norte	61.683266	125.333333	41.986	
1	6015	United States	Del Norte	8.809403	56.666667	41.893	
2	6015	United States	Del Norte	11.803890	285.625000	41.921	
3	6015	United States	Del Norte	13.337636	607.909091	41.950	
4	6015	United States	Del Norte	37.876299	630.000000	41.978	
...	...	...	...	...	...	...	
2755	6025	United States	Imperial	251.184279	105.214286	32.763	
2756	6025	United States	Imperial	263.129792	78.937500	32.783	
2757	6025	United States	Imperial	21.180429	31.000000	32.647	
2758	6025	United States	Imperial	27.895799	32.428571	32.550	
2759	6025	United States	Imperial	32.676584	36.333333	32.453	

	...	mean_lcoe	mean_res	reg_mult	reinforcement_cost_per_mw_ac	\
0	...	50.717754	4.120177	1.081992	1.422906e+06	
1	...	52.247262	4.062002	1.084387	1.781207e+06	
2	...	50.355025	4.175112	1.083833	1.781207e+06	
3	...	48.237379	4.398418	1.083631	1.695098e+06	
4	...	48.489174	4.466510	1.083503	1.905410e+06	
...	...	...	...	...	...	
2755	...	33.831718	5.981142	1.006305	6.154248e+04	
2756	...	33.992068	5.928655	1.001099	6.154248e+04	
2757	...	34.355245	5.942000	1.008530	8.635921e+05	
2758	...	33.978806	5.987235	1.007868	7.963230e+05	
2759	...	33.505549	6.063752	1.007602	7.963230e+05	

	reinforcement_dist_km	sc_point_gid	state	timezone	total_lcoe	\
0	376.101552	25466	California	-8	639.162709	
1	256.980915	25846	California	-8	160.864954	
2	256.980915	25847	California	-8	126.485579	
3	241.641441	25848	California	-8	128.883800	
4	273.703922	25849	California	-8	4044.898673	
...	...	...	...	...	...	
2755	8.724701	66550	California	-8	80.468533	
2756	8.724701	66551	California	-8	79.295824	
2757	310.095260	66929	California	-7	65.388867	
2758	302.103350	67309	California	-7	61.994083	
2759	302.103350	67689	California	-7	64.855774	

trans_cap_cost_per_mw_ac

```

0          1.429827e+07
1          1.040914e+06
2          2.702859e+05
3          5.731362e+05
4          1.099016e+08
...
2755       1.697601e+06
2756       1.632027e+06
2757       2.912543e+05
2758       2.571834e+05
2759       3.989861e+05

```

[2760 rows x 24 columns]

```
[40]: df.head()
```

```

[40]:      _id  area_sq_km  capacity_mw_ac  capacity_mw_dc  \
0  6a23b30b710edabdd297d8ea  0.203472      6.529325      8.749296
1  6a23b30b710edabdd297da12  0.684612     21.968892     29.438317
2  6a23b30b710edabdd297da13  1.606716     51.558794     69.088788
3  6a23b30b710edabdd297da14  1.082808     34.746824     46.560747
4  6a23b30b710edabdd297da15  0.016524      0.530248      0.710532

      cnty_fips  country  county  dist_km  elevation  latitude  ...  \
0         6015  United States  Del Norte  61.683266  125.333333    41.986  ...
1         6015  United States  Del Norte   8.809403   56.666667    41.893  ...
2         6015  United States  Del Norte  11.803890  285.625000    41.921  ...
3         6015  United States  Del Norte  13.337636  607.909091    41.950  ...
4         6015  United States  Del Norte  37.876299  630.000000    41.978  ...

      mean_lcoe  mean_res  reg_mult  reinforcement_cost_per_mw_ac  \
0  50.717754  4.120177  1.081992      1.422906e+06
1  52.247262  4.062002  1.084387      1.781207e+06
2  50.355025  4.175112  1.083833      1.781207e+06
3  48.237379  4.398418  1.083631      1.695098e+06
4  48.489174  4.466510  1.083503      1.905410e+06

      reinforcement_dist_km  sc_point_gid  state  timezone  total_lcoe  \
0          376.101552      25466  California      -8    639.162709
1          256.980915      25846  California      -8    160.864954
2          256.980915      25847  California      -8    126.485579
3          241.641441      25848  California      -8    128.883800
4          273.703922      25849  California      -8   4044.898673

      trans_cap_cost_per_mw_ac
0          1.429827e+07
1          1.040914e+06

```

```

2          2.702859e+05
3          5.731362e+05
4          1.099016e+08

```

[5 rows x 24 columns]

```
[42]: df.tail()
```

```

[42]:
      _id  area_sq_km  capacity_mw_ac  capacity_mw_dc  \
2755  6a23b311710edabdd2985cf7    65.553945    2103.596587    2818.819621
2756  6a23b311710edabdd2985cf8    59.255064    1901.468099    2547.967732
2757  6a23b311710edabdd2985df7     2.258280     72.467186     97.106036
2758  6a23b311710edabdd2985ef5    13.585076    435.939039    584.158284
2759  6a23b311710edabdd2985fec     3.486564    111.882284    149.922253

      cnty_fips  country  county  dist_km  elevation  latitude  \
2755      6025  United States  Imperial  251.184279  105.214286    32.763
2756      6025  United States  Imperial  263.129792   78.937500    32.783
2757      6025  United States  Imperial   21.180429   31.000000    32.647
2758      6025  United States  Imperial   27.895799   32.428571    32.550
2759      6025  United States  Imperial   32.676584   36.333333    32.453

      ...  mean_lcoe  mean_res  reg_mult  reinforcement_cost_per_mw_ac  \
2755  ...   33.831718   5.981142   1.006305                61542.478002
2756  ...   33.992068   5.928655   1.001099                61542.478002
2757  ...   34.355245   5.942000   1.008530                863592.131349
2758  ...   33.978806   5.987235   1.007868                796323.046671
2759  ...   33.505549   6.063752   1.007602                796323.046671

      reinforcement_dist_km  sc_point_gid  state  timezone  total_lcoe  \
2755              8.724701          66550  California        -8    80.468533
2756              8.724701          66551  California        -8    79.295824
2757             310.095260          66929  California        -7    65.388867
2758             302.103350          67309  California        -7    61.994083
2759             302.103350          67689  California        -7    64.855774

      trans_cap_cost_per_mw_ac
2755          1.697601e+06
2756          1.632027e+06
2757          2.912543e+05
2758          2.571834e+05
2759          3.989861e+05

```

[5 rows x 24 columns]

```
[38]: df.shape
```

```
[38]: (2760, 24)
```

```
[26]: df.columns
```

```
[26]: Index(['_id', 'area_sq_km', 'capacity_mw_ac', 'capacity_mw_dc', 'cnty_fips',  
        'country', 'county', 'dist_km', 'elevation', 'latitude', 'lcot',  
        'longitude', 'mean_cf_ac', 'mean_cf_dc', 'mean_lcoe', 'mean_res',  
        'reg_mult', 'reinforcement_cost_per_mw_ac', 'reinforcement_dist_km',  
        'sc_point_gid', 'state', 'timezone', 'total_lcoe',  
        'trans_cap_cost_per_mw_ac'],  
        dtype='object')
```

```
[30]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2760 entries, 0 to 2759  
Data columns (total 24 columns):  
#   Column                                Non-Null Count  Dtype  
---  -  
0   _id                                    2760 non-null   object  
1   area_sq_km                            2760 non-null   float64  
2   capacity_mw_ac                        2760 non-null   float64  
3   capacity_mw_dc                        2760 non-null   float64  
4   cnty_fips                             2760 non-null   int64  
5   country                               2760 non-null   object  
6   county                                2760 non-null   object  
7   dist_km                               2760 non-null   float64  
8   elevation                             2760 non-null   float64  
9   latitude                              2760 non-null   float64  
10  lcot                                  2760 non-null   float64  
11  longitude                             2760 non-null   float64  
12  mean_cf_ac                            2760 non-null   float64  
13  mean_cf_dc                            2760 non-null   float64  
14  mean_lcoe                             2760 non-null   float64  
15  mean_res                              2760 non-null   float64  
16  reg_mult                              2760 non-null   float64  
17  reinforcement_cost_per_mw_ac          2760 non-null   float64  
18  reinforcement_dist_km                 2760 non-null   float64  
19  sc_point_gid                          2760 non-null   int64  
20  state                                 2760 non-null   object  
21  timezone                              2760 non-null   int64  
22  total_lcoe                            2760 non-null   float64  
23  trans_cap_cost_per_mw_ac              2760 non-null   float64  
dtypes: float64(17), int64(3), object(4)  
memory usage: 517.6+ KB
```

```
[32]: df.isnull().sum()
```

```
[32]: _id 0
      area_sq_km 0
      capacity_mw_ac 0
      capacity_mw_dc 0
      cnty_fips 0
      country 0
      county 0
      dist_km 0
      elevation 0
      latitude 0
      lcot 0
      longitude 0
      mean_cf_ac 0
      mean_cf_dc 0
      mean_lcoe 0
      mean_res 0
      reg_mult 0
      reinforcement_cost_per_mw_ac 0
      reinforcement_dist_km 0
      sc_point_gid 0
      state 0
      timezone 0
      total_lcoe 0
      trans_cap_cost_per_mw_ac 0
      dtype: int64
```

```
[34]: df.describe()
```

```
[34]:
```

	area_sq_km	capacity_mw_ac	capacity_mw_dc	cnty_fips	dist_km \
count	2760.000000	2760.000000	2760.000000	2760.000000	2760.000000
mean	10.905117	349.940318	468.920051	6057.818116	68.918144
std	15.730126	504.772674	676.395437	29.817256	80.246043
min	0.000324	0.010397	0.013932	6001.000000	0.217279
25%	0.759456	24.370601	32.656611	6029.000000	14.075339
50%	3.441042	110.421494	147.964803	6057.000000	31.306847
75%	15.002416	481.420748	645.103885	6081.000000	100.551807
max	98.033984	3145.865672	4215.461295	6115.000000	522.221655

	elevation	latitude	lcot	longitude	mean_cf_ac \
count	2760.000000	2760.000000	2760.000000	2760.000000	2760.000000
mean	755.161993	37.320631	905.512590	-119.899466	0.317791
std	635.452926	2.630430	9564.095750	2.414509	0.024384
min	-67.700000	32.453000	4.818287	-124.376000	0.233982
25%	198.822802	35.080750	47.174913	-121.727500	0.300482
50%	594.750000	37.229000	69.938112	-120.304000	0.314997
75%	1229.791667	39.621250	124.397697	-118.089500	0.338322
max	2668.000000	41.998000	317542.381412	-114.205000	0.365014

	mean_cf_dc	mean_lcoe	mean_res	reg_mult \
count	2760.000000	2760.000000	2760.000000	2760.000000
mean	0.237157	40.027005	5.362782	1.094312
std	0.018197	3.681153	0.410331	0.029894
min	0.174613	33.439612	3.995288	0.996734
25%	0.224241	37.039122	5.059412	1.082489
50%	0.235072	40.491866	5.335262	1.094145
75%	0.252479	42.443800	5.697159	1.108492
max	0.272398	53.886997	6.129000	1.197262

	reinforcement_cost_per_mw_ac	reinforcement_dist_km	sc_point_gid \
count	2.760000e+03	2760.000000	2760.000000
mean	1.615526e+06	219.216679	46057.171377
std	1.193975e+06	157.570814	11248.656686
min	1.251062e+04	1.483675	25466.000000
25%	5.442686e+05	87.576130	36107.750000
50%	1.635948e+06	195.228389	46374.500000
75%	2.318046e+06	328.005401	55508.250000
max	6.181621e+06	992.546246	67689.000000

	timezone	total_lcoe	trans_cap_cost_per_mw_ac
count	2760.000000	2760.000000	2.760000e+03
mean	-7.998188	945.539595	3.065064e+07
std	0.042532	9563.946920	3.419234e+08
min	-8.000000	43.944464	1.881313e+04
25%	-8.000000	85.917675	4.336908e+05
50%	-8.000000	109.871546	9.334887e+05
75%	-8.000000	165.616347	1.916579e+06
max	-7.000000	317582.748783	1.096335e+10

2.2 Exploritory Plots

```
[41]: fig, axes = plt.subplots(1, 2, figsize=(14, 14))
fig.suptitle('LCOE by State', fontsize=14, y=1.01)

lcoe_order = df.groupby('county')['mean_lcoe'].mean().sort_values().index

# left: barplot with error bars
# - left: barplot by county
ax = axes[0]

# Calculate mean LCOE per county in order
county_means = df.groupby('county')['mean_lcoe'].mean().reindex(lcoe_order)

ax.barh(range(len(lcoe_order)), county_means.values, color='steelblue',
        height=0.7)
```

```

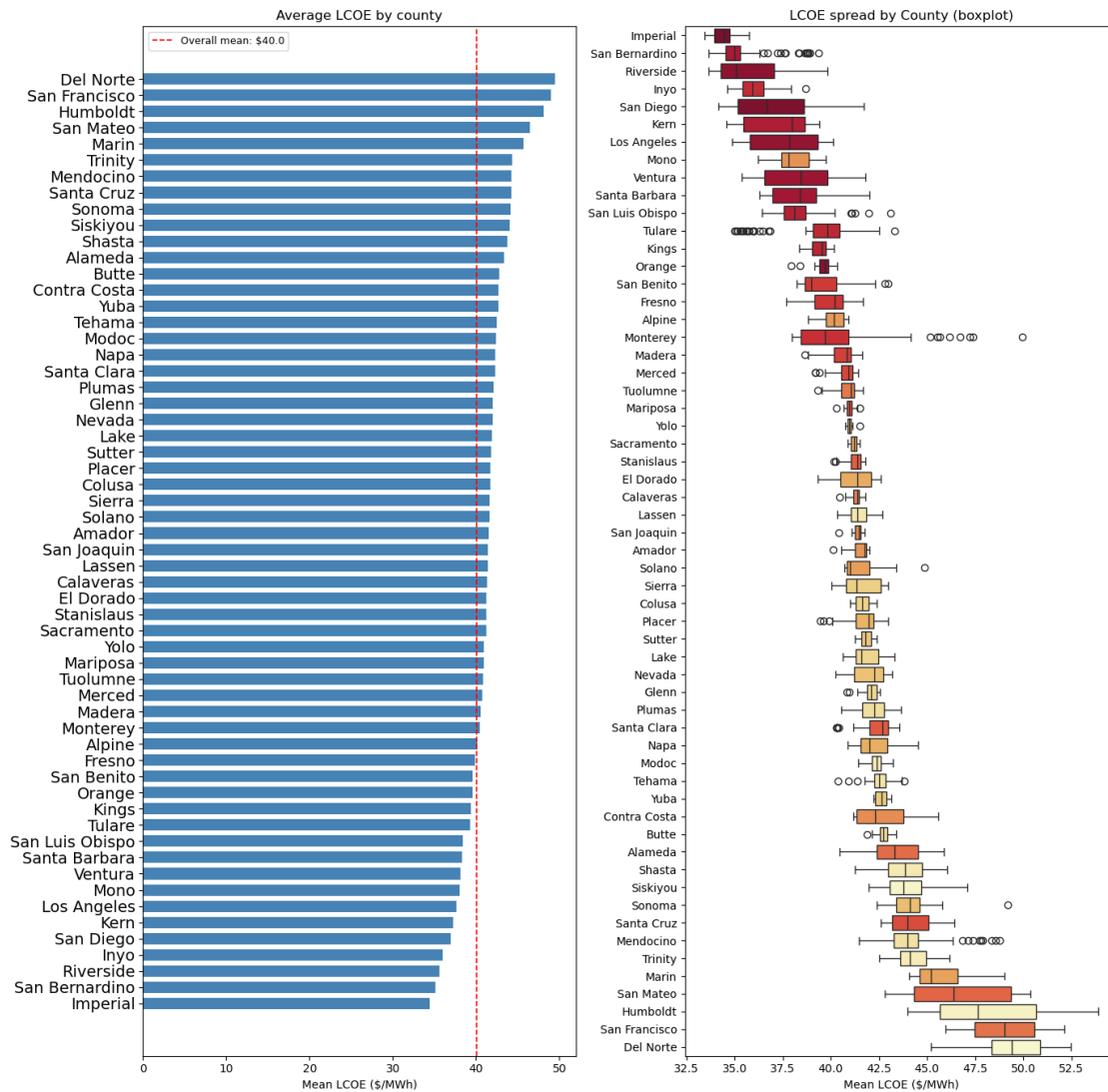
ax.set_yticks(range(len(lcoe_order)))
ax.set_yticklabels(lcoe_order, fontsize=14)
ax.axvline(df['mean_lcoe'].mean(), color='red', linestyle='--', linewidth=1.2,
           label=f"Overall mean: ${df['mean_lcoe'].mean():.1f}")
ax.set_xlabel('Mean LCOE ($/MWh)')
ax.set_ylabel('')
ax.set_title('Average LCOE by county')
ax.legend(fontsize=9)

#   right: boxplot shows spread
ax2 = axes[1]
sns.boxplot(data=df, x='mean_lcoe', y='county', order=lcoe_order,
            hue='county', palette='YlOrRd', legend=False)
ax2.set_xlabel('Mean LCOE ($/MWh)')
ax2.set_ylabel('')
ax2.set_title('LCOE spread by County (boxplot)')

plt.tight_layout()
plt.savefig('plot4_lcoe_by_state.png', bbox_inches='tight')
plt.show()

```

LCOE by State



Del Norte, San Francisco and Humboldt Counties have the highest LCOE Averages indicating that total cost is higher to build in these counties. Imperial, San Bernardino, and Riverside have the lowest LCOE Averages indicates that the total cost to build is likely more favorable here.

San Berdaino, Tulare, Mendocino and Monterey have a number of outliers suggesting that there are more specific areas within the couties that have much lower or higher LCOE scores. This could suggest variability in site suitability for the area.

```
[46]: fig, ax = plt.subplots(figsize=(10, 8))
fig.suptitle('Correlation Heatmap of Numeric Features', fontsize=14,
fontweight='bold')

num_cols = [
```

```

    'mean_cf_ac', 'mean_lcoe', 'capacity_mw', 'area_sq_km',
    'dist_km', 'lcot', 'elevation', 'reg_mult',
    'reinforcement_cost_per_mw_ac', 'total_lcoe'
]

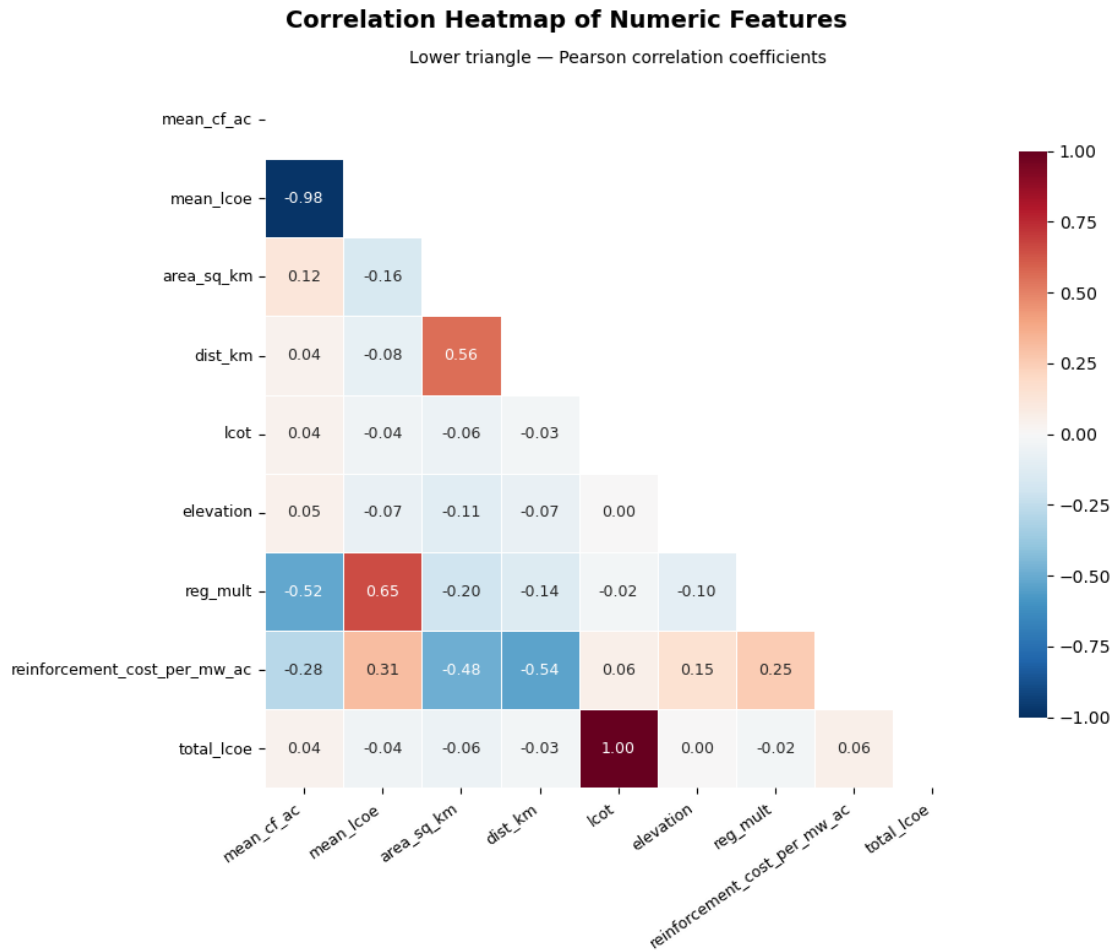
# only keep columns that exist in your dataframe
num_cols = [c for c in num_cols if c in df.columns]
corr = df[num_cols].corr()

# lower triangle mask
mask = np.triu(np.ones_like(corr, dtype=bool))

sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, vmin=-1, vmax=1,
            square=True, linewidths=0.5,
            cbar_kws={'shrink': 0.8}, ax=ax,
            annot_kws={'size': 9})
ax.set_xticklabels(ax.get_xticklabels(), rotation=35, ha='right', fontsize=9)
ax.set_yticklabels(ax.get_yticklabels(), rotation=0, fontsize=9)
ax.set_title('Lower triangle - Pearson correlation coefficients', fontsize=10,
            pad=10)

plt.tight_layout()
plt.savefig('plot7_correlation_heatmap.png', bbox_inches='tight')
plt.show()

```



Mean_af_ac (Capacity Factor) & Mean_lcoe(Avg. Levelized Cost of Energy) have a negative correlation score of -0.98 suggesting that sites with higher solar energy outputs lower the overall cost of energy.

reg_mult (regional cost multiplier) and reinforcement_cost_per_mw_ac (cost to upgrade the existing grid infrastructure) have the highest correlation scores suggesting what region of the county and how far from the line to be most closely correlated with high LCOE scores

3 Machine Learning

4 Linear Regression

```
[122]: from sklearn.datasets import fetch_california_housing
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```

[124]: # X = df.data # Features
        # y = df.target # Target variable

[126]: X =
        ↪df[['mean_cf_ac', 'mean_cf_dc', 'dist_km', 'elevation', 'reg_mult', 'reinforcement_cost_per_mw_a

[128]: y = df['mean_lcoe']

[130]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
        ↪random_state=42)

[132]: model = LinearRegression()

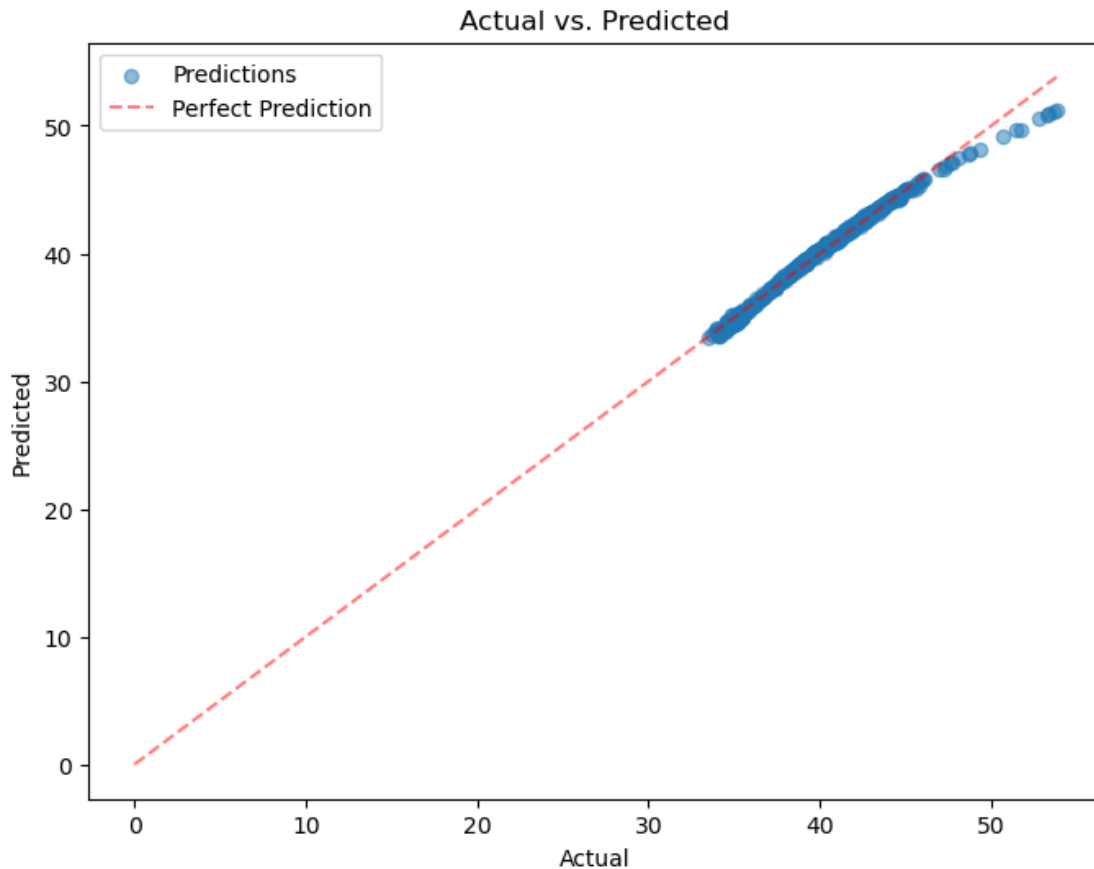
[134]: model.fit(X_train, y_train)

[134]: LinearRegression()

[136]: y_pred = model.predict(X_test)

[138]: plt.figure(figsize=(8, 6))
        plt.scatter(y_test, y_pred, alpha=0.5, label='Predictions')
        plt.plot([0, max(y_test)], [0, max(y_test)], '--r', alpha=0.5, label='Perfect
        ↪Prediction')
        plt.title('Actual vs. Predicted')
        plt.xlabel('Actual')
        plt.ylabel('Predicted')
        plt.legend()
        plt.show()

```



```
[140]: mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

Mean Squared Error: 0.13650324005314005

```
[142]: from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print(f"MAE: {mae:.4f}")
print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
```

MAE: 0.2303

MSE: 0.1365

RMSE: 0.3695

The Actual vs prediction line is closely aligned suggesting that the linear regression model is highly

accurate. With slightly less accuracy near the higher LCOE scores

MAE of 0.23 indicates that on average the models prediction were off by \$0.23 LCOE

MSE of 0.1365 indicates a low squared error, meaning no large outliers in the predictions

RMSE of 0.3695 which means the average prediction error is around \$0.37 LCOE

5 K-Nearest Neighbors (KNN)

```
[173]: from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- Normalize Features ---
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # fit + transform on train
X_test_scaled = scaler.transform(X_test)        # transform only on test

# --- KNN Regression ---
knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X_train_scaled, y_train)
y_pred_knn = knn_reg.predict(X_test_scaled)

# --- Metrics ---
mae = mean_absolute_error(y_test, y_pred_knn)
mse = mean_squared_error(y_test, y_pred_knn)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_knn)

print("KNN Regression Results (k=5, normalized)")
print(f" MAE:  {mae:.4f}")
print(f" MSE:  {mse:.4f}")
print(f" RMSE: {rmse:.4f}")
print(f" R²:   {r2:.4f}")

# --- Actual vs Predicted Plot ---
plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_test, y=y_pred_knn, alpha=0.5, color='steelblue')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=1.5, label='Perfect Fit')
plt.xlabel("Actual LCOE ($/MWh)")
plt.ylabel("Predicted LCOE ($/MWh)")
```



```
plt.title("KNN Regression - Actual vs Predicted LCOE (NREL Dataset, Normalized)")
plt.legend()
plt.tight_layout()
plt.show()
```

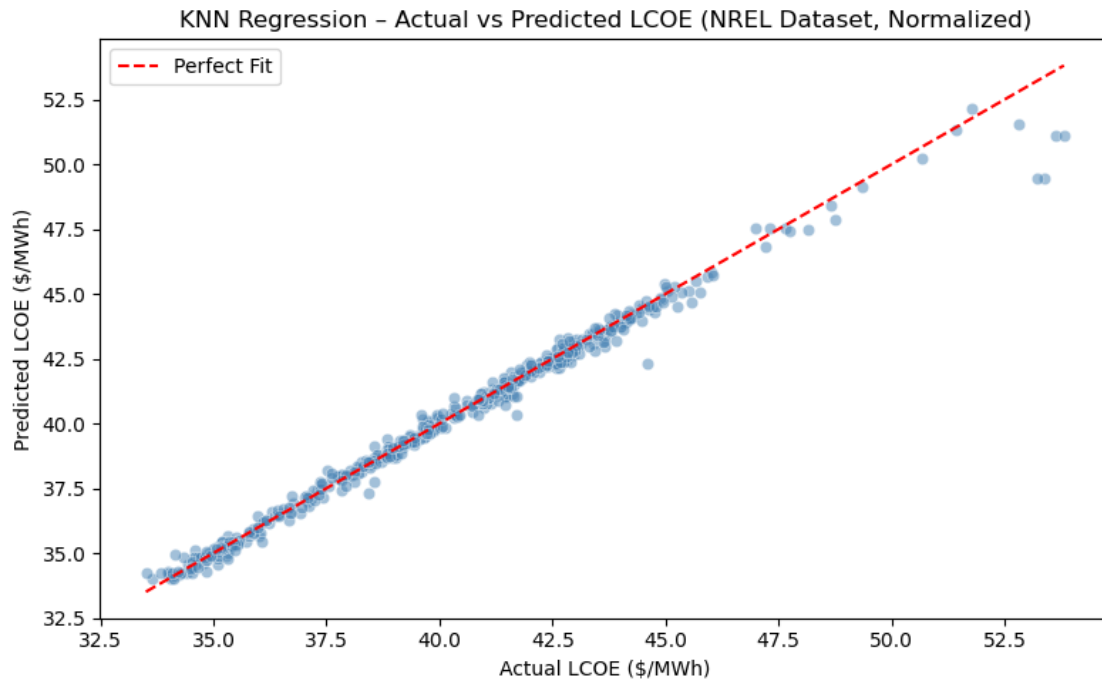
KNN Regression Results (k=5, normalized)

MAE: 0.2100

MSE: 0.1542

RMSE: 0.3927

R^2 : 0.9883



5.0.1 Conclusions

Both models perform well overall

KNN has a slightly lower MAE (better on average error) but higher RMSE (worse on larger errors) — meaning KNN is more consistent in the middle range but linear regression handles outliers better

5.0.2 References

- Academic (if any)
- Online (if any)

Module 13 Example 4: House Price Prediction using KNN and GBR

Module 13 Example 3: Linear Regression to predict MPG and Car Prices

Module 13 Example 6: MongoDB - NBA dataset

5.0.3 Credits

- If you use and/or adapt your code from existing projects, you must provide links and acknowledge the authors. > *This code is based on ... (if any)*

<https://github.com/lukalee/CISD-43-Big-Data-Modeling-Analysis/blob/11f9e916a3e0fed154f3a4ecec6934654b9dd%20Project-%20Code.ipynb>

```
[ ]: # End of Project
```